



A Trust Calculus for Attestation Tiers: Computing Residual Trust and Testing Whether Deployments Can Be Ranked

— Which parties must you still trust, and for how long would you not know?

Jim Schwoebel^{1,2}

¹ Quome ² Advanced AI Society, <https://advancedaisociety.org>¹

ABSTRACT

Attestation systems are routinely described as removing the need to trust anybody. The Proof-of-Control standard [1] says of its Tier 3 that “there is nobody left to trust; anyone can check the evidence with published tools.” That sentence is false, and this paper shows it is false rather than suspecting it. Deploying on Intel TDX and enumerating what a verifier must accept to believe one measurement yields five distinct parties, and four of the five have no detection mechanism at all: if they misbehave, nothing in the system will ever say so. The fifth is bounded only by how often revocation data is refreshed. We give a small calculus that computes this set instead of arguing about it. A judgment $\Gamma \vdash P : T, \Delta$ reads “proposition P holds provided the parties in T are honest, and a violation would be detected within Δ ”; mechanisms compose by joining T under union and Δ under max, the former over a trust-set lattice of finite height. We implement it in `parallax`, a Rust tool that reads a deployment description and prints the residual trust set, and we show that termination on cyclic delegation — the case the original design proposed to handle with tabled resolution — follows from the finite height of that one lattice and needs no tabling. Of the deployments we encoded, the two pairs that attest the same proposition and can therefore be compared at all come out *incomparable* under set inclusion rather than merely different, which would mean no ordinal tier ladder can rank them. We then run each of the four ways we can think of to order deployments by verifiability — our own list, not a survey — against those same encodings. All four fail, and each fails differently: ordering by the size of the trust set ranks a bare software host above a witness quorum, which inverts the word. We are explicit about the limit that matters most: the solver computes over the dependencies somebody wrote down, and cannot discover one that was left out.

Artifact. `parallax` [2] is Apache-2.0. Every *table and figure* in Sections 5 and 6 is regenerated from the tool by `scripts/regen-results.sh` and read into this document from the generated files; continuous integration fails if a committed result disagrees with the code that produced it. Sentences near those tables sometimes restate a figure from one, and those sentences are typed rather than generated; the ones that could drift are pinned by tests named in `paper/README.md`. Figures elsewhere in the paper — deployment parameters, code size, the size of the termination example — are stated in the prose and are not tool output.

¹Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript; additional co-authors will be added only upon explicit consent and substantive contribution.

Contents

1	The Problem	3
2	What Others Have Built	5
3	Writing It Down Precisely	6
3.1	The judgment	6
3.2	The two lattices	7
3.3	The composition rules	7
3.4	Delegation, and why it is a fixpoint	9
3.5	Provenance, and why the comparison is order-independent	9
4	Why It Terminates	10
5	Does It Actually Work?	11
5.1	Reproducing the hand-derived TDX set	11
5.2	The comparison matrix	12
5.3	The shared dependency	12
5.4	Detection latency and policy	13
6	Can the Ladder Be Repaired?	15
7	What We Still Don't Know	18
7.1	The completeness problem, which is the real objection	18
7.2	The rules are not proved sound	18
7.3	Principal independence is asserted, never computed	18
7.4	Whether Δ should compose as a max	19
7.5	The evidence base is five deployments, all ours	19
7.6	If the ladder goes, what replaces it	19
8	Conclusion	19

1 The Problem

Somebody hands you a signed attestation. It says a particular piece of software, with a particular measurement, ran inside a hardware-protected environment on a machine you do not own [3]. The pitch is that you no longer have to take anyone’s word for it: the hardware vouches for the software, the signature vouches for the hardware, and you can check the whole thing yourself with published tools.

So check it. Here is what you actually did.

You verified a signature chain rooted in a key that Intel provisioned. You believed that key was issued to real silicon, because Intel’s certification service said so. You believed the quote was produced by a quoting enclave that measured the domain honestly and did not leak its signing key. You believed the host machine did not inject a different firmware measurement at boot. And you believed that the reference value you compared against — the number that says “this measurement means *that* code” — was published by somebody who knows what code it belongs to.

That is five parties. You did not remove trust; you moved it. And the thing that should stop the conversation is not the count but the fourth column of Table 1: for four of those five, there is no detection mechanism at all. If Intel’s silicon is subverted, or the quoting enclave’s key leaks, or the host injects a firmware measurement of its own choosing, or the reference values are wrong, the attestation still verifies cryptographically. Nothing anywhere in the system contradicts it. The failure is silent and it is permanent. The only one of the five you would ever hear about is Intel’s certification service, and only because its collateral goes stale on a schedule.

Table 1: What a verifier must accept to believe a single Intel TDX measurement, derived by deploying the Proof-of-Control reference implementation and enumerating what was left. The fourth column is the point.

Party	What you must assume	What a breach buys the adversary	Detectable in
Intel	the TDX module, microcode and CPU behave as specified; the provisioning key is not misissued	arbitrary quote forgery, state extraction, memory manipulation inside the hardware boundary	never
Intel’s certification service (PCS)	the collateral fetched to check a quote is authentic (TCB recovery info, CRLs, QE certs)	revoked, vulnerable or emulated hardware presented as a valid target	collateral refresh interval (≈ 12 h)
The quoting enclave	it measures TD reports accurately and signs without key leakage	forged execution claims on a compromised host	never
The cloud operator	the host does not extract domain memory outside the TDX threat model; the firmware measured into RTMRs is what it claims	deceptive measurement injection at boot; side-channel extraction	never
Whoever publishes reference values	the MRTD compared against belongs to the code you believe it does	valid attestation of subverted software inside a genuine TD	never

Anchoring the evidence log to a public ledger removes none of these. It is a real and valuable

property — it makes the *history* publicly auditable — but the *measurement* still rests on the chain above. And, as Section 3 shows, anchoring does not even come for free: it ingests a new assumption about the settlement layer’s execution fidelity while it is converting the old one.

Two properties that are being conflated. The tier ladder runs together two things a careful reader will separate:

- **Public verifiability** — anyone may check the evidence, with published tools, without permission from the party being checked.
- **Trust independence** — no named party’s honesty is load-bearing for the conclusion.

Hardware attestation buys the first outright. It leaves the second largely intact. Confusing them is what produces a sentence like “there is nobody left to trust,” and that sentence sits at the binary threshold the whole standard is organized around. If the threshold is mis-drawn, every conformance claim above it says less than its definition promises.

Arguing about this is not working. Everyone in the field agrees, when pressed, that hardware roots are assumptions. What nobody has is a way to settle a specific disagreement about a specific deployment. Reviewer says “you are still trusting the build pipeline”; vendor says “no, the attestation covers that”; and there the matter rests, because there is no procedure that takes a system description and returns an answer. That is what we set out to build.

What we did. We wrote down a judgment form, gave composition rules for each mechanism the standard admits, implemented them, and ran them on five encoded deployments.

1. **The calculus** (Section 3). A judgment $\Gamma \vdash P : T, \Delta$: the proposition P holds provided the parties in T are honest, with any violation detectable within Δ . Composition is join in two lattices — union on trust sets, max on detection latencies.
2. **The tool** (Section 3, Section 5). **parallax**, about 1 800 lines of Rust excluding tests and roughly twice that again in tests, reads a deployment as TOML and emits the residual trust set, a machine-readable manifest, a comparison between two deployments, a shared-dependency analysis, and a policy check.
3. **Termination, the easy way** (Section 4). The original design proposed tabled (SLG) resolution so that circular delegation would terminate. It is not needed. The fixpoint lives in the powerset of a finite assumption set, which has finite height, so bottom-up evaluation halts regardless of cycles. We say why the harder machinery solves a problem we do not have.
4. **What we found** (Section 5). The tool reproduces Table 1 — each party’s capability, detection bound and impact, and the fact that exactly these five are load-bearing — from a deployment description that says none of it, and it finds a shared build-pipeline dependency underneath a hybrid TEE-plus-ZK design that is marketed as defence in depth. Of the encoded deployments, the two pairs that can be compared at all — comparison requires both sides to attest the same proposition — come out *incomparable*.
5. **Whether the ladder can be repaired** (Section 6). Four candidate orderings, run against the four non-hybrid deployments. All four fail, each in its own way: counting assumptions ranks the systems backwards, set inclusion ranks almost nothing, composed detection latency is ∞ for every deployment and so ranks everything the same, and collusion cost is not derivable from a deployment description at all.

6. **What is wrong with all of this** (Section 7). Led by the completeness problem, which we think is the strongest objection to the whole approach and which we have not solved.

One thing we are not claiming. `parallax` has been run on five deployments, all of which we wrote. It reproduces a trust set we had already derived by hand, and it found a shared dependency in a deployment we constructed. It has not been pointed at somebody else’s production system, and no claim in this paper should be read as saying it has. Section 5 says exactly which examples exist and Section 7 says what that leaves open.

2 What Others Have Built

The roles exist; the obligations do not. The IETF’s remote attestation architecture, RFC 9334 [4], already gives us the vocabulary: Attester, Verifier, Relying Party, Endorser, Reference Value Provider. This is the closest existing thing and we adopt its names. But RFC 9334 is *descriptive*. It says who plays what part in an attestation exchange. It does not say what breaks when a part is played dishonestly, and that — not the role taxonomy — is what a relying party needs. Reading RFC 9334 tells you an Endorser exists. It does not tell you that if the Endorser lies, nothing anywhere in the protocol will ever notice.

Authorization logics: the right primitives, the wrong direction. Abadi, Burrows, Lampson and Plotkin’s calculus for access control [5], and the Taos work that grew from it [6], gave us **says** and **speaks-for**. Those are exactly the right shape for “Intel says this measurement,” and our delegation relation is **speaks-for** under another name. Nexus authorization logic [7] is the closest prior art of all: an authorization logic built specifically for attestation-based systems. NAL divides the bases for believing a claim into three kinds — *axiomatic* (believed by policy fiat), *analytic* (derived by mechanical analysis of code), and *synthetic* (produced by a trusted transformation or an isolated execution environment). That is already a taxonomy of *why* a claim is believed, and it maps onto what we compute: every assumption in a residual trust set is, in NAL’s terms, an axiomatic or synthetic basis that survived. What NAL does not do is emit the set.

Nor do the declarative policy languages. SecPAL [8], DKAL [9] and Binder [10] evaluate an authorization query against a fixed trust root and return yes or no. None of them inverts the question. Given a query that succeeded, none will tell you the minimal set of principals whose honesty the success depended on. That inversion is the technical core of this paper, and as far as we can find it has not been done — not because it is hard in these systems, but because the systems were built for a different job. Their trust root is a given; ours is the answer.

Transparency converts trust; it does not remove it. Certificate Transparency [11, 12] and CONIKS [13] are the canonical demonstrations that you can take a party you had to trust unconditionally and turn it into a party whose misbehaviour will be noticed within a bounded time. This is a genuine and large win, and it is also not the same win as removing the party. That distinction — conversion, not removal — is the one the tier ladder most needs and most lacks, and it is why detection latency is a first-class term in our judgment rather than a footnote. A log operator with a 15 min gossip bound is still in your trust set. It is in there with a number next to it.

Hardware roots are assumptions, and the attacks say so precisely. Foreshadow [14], SGAXe [15], Plundervolt [16] and ÆPIC Leak [17], alongside Costan and Devadas’s architectural

analysis [18], establish that enclave isolation is a property that has repeatedly failed. The shape of these attacks matters more than their existence: they break isolation *without* invalidating the signing key. The quote remains cryptographically valid while the claim it makes is false. Any calculus that treats “the signature verified” as discharging the hardware assumption is therefore modelling the wrong thing, which is why the endorser appears in our output with $\Delta = \infty$ rather than not appearing at all.

Restructuring trust cryptographically. Direct Anonymous Attestation [19] is worth naming because it shows how much of a trust structure can be moved rather than merely documented. We do not do that here; we compute over the structure as built.

Tabled resolution, which we ended up not needing. SLG resolution [20] and XSB [21] are the standard answer to a logic program that diverges on cyclic data, and the design this paper implements originally specified them for exactly that reason: real deployments contain mutual and circular attestation, and plain SLD resolution does not terminate on them. Section 4 explains why we did not need it.

3 Writing It Down Precisely

Sentences like “you still have to trust the vendor” are fine until two people disagree about what they cover, so here it is in symbols. Everything in this section is what the code does; the composition rules below are transcribed from `src/mechanism.rs` and `src/solve.rs` in the artifact, not drafted for the paper.

3.1 The judgment

Let $\mathcal{P}$ be a finite set of named principals and $\mathcal{C}$ a finite set of capability names — “signs quotes honestly,” “publishes correct reference values,” and so on. A *detection latency* is either a finite bound or nothing at all:

$$\Delta \in \mathcal{L} = \{ \text{Bounded}(t) : t \in \mathbb{N} \} \cup \{ \infty \}. \quad (1)$$

∞ is not a large number. It means there is no detection mechanism, so the violation is silent forever. Keeping it as a distinct symbol rather than a big integer is deliberate: it is the difference this paper exists to make visible.

An *assumption* is a four-tuple plus a provenance tag,

$$a = (p, c, \Delta, \iota, m) \in \mathcal{P} \times \mathcal{C} \times \mathcal{L} \times \mathcal{I} \times \mathcal{M}, \quad (2)$$

read as: *principal p must be honest about capability c ; a violation would be detected within Δ ; a breach buys the adversary ι ; and this assumption entered because of mechanism m .* The impact ι ranges over SOUNDNESS (a false claim can be made to verify), REVOCATION (revoked or vulnerable components pass as valid), SPLITVIEW (equivocation is possible, but leaves contradictory evidence), and AVAILABILITY (evidence can be withheld, not forged). A *residual trust set* T is a finite set of assumptions.

The judgment is

$$\Gamma \vdash P : T, \Delta \quad (3)$$

— *proposition P holds provided every party in T is honest about the capability named, and any violation is detected within Δ .* In the implementation Γ is the deployment description: the declared

principals, the mechanisms and their parameters, and the delegation edges. P is the deployment’s single `claim` string. Δ is derived from T , not carried alongside it: $\Delta = \bigsqcup_{a \in T} \Delta_a$.

3.2 The two lattices

Latency. $\mathcal{L}$ is ordered with every bound below ∞ and finite bounds ordered by magnitude. `Join` takes the worse case and ∞ absorbs:

$$\Delta_1 \sqcup \Delta_2 = \begin{cases} \infty & \text{if } \Delta_1 = \infty \text{ or } \Delta_2 = \infty, \\ \text{Bounded}(\max(t_1, t_2)) & \text{otherwise.} \end{cases} \quad (4)$$

`Meet` takes the better case, with ∞ as the identity.

$\mathcal{L}$ has *infinite* height: $\text{Bounded}(0) < \text{Bounded}(1) < \dots$ is an infinite ascending chain below ∞ . This matters for Section 4 and it is not a problem, because the fixpoint does not iterate here. Δ is a fold over the finished trust set, computed once after the fixpoint has converged; the only lattice the solver iterates in is the trust-set lattice, whose height is finite. Confusing the two would make the termination argument false, so it is worth stating which one carries it.

The reading of `join` is that a system is only as detectable as its least detectable part. Whether that is the right reading is an open question and Section 7 returns to it. The empty trust set has $\Delta = \text{Bounded}(0)$: nothing to detect.

Trust sets. $(\mathcal{T}, \subseteq)$ is the powerset of the assumptions derivable from a deployment, ordered by inclusion, with `join` = union and `meet` = intersection. This is a *partial* order and that is the load-bearing fact of the paper: two trust sets can be *incomparable*, each containing an assumption the other lacks. When that happens neither deployment is “more verifiable” than the other, and no ordinal tier can rank them.

We wrote the order by hand rather than deriving it, and the reason is worth recording because it is a trap. Rust’s derived ordering on an ordered set type is lexicographic and *total*: under it $\{2\}$ is not $\leq \{1, 2\}$, so `join` could return a value that is not above its own operands, violating the lattice contract in a way that produces wrong answers rather than a crash.

3.3 The composition rules

Each mechanism contributes the assumptions a verifier must accept in order to believe a claim that mechanism supports. Write $\mathcal{A}(m)$ for that contribution. All latencies below are the mechanism’s declared parameters, parsed as durations; **never** is accepted and means ∞ .

Three of these deserve a sentence each.

tee_attestation is Table 1. The five rows of the rule are the five rows of the TDX table, and only the collateral authority carries a bound. This is the sense in which the tool reproduces the hand-derived result: the deployment file declares five principal identifiers, a refresh interval and a mechanism kind, and the five-party set — with each party’s capability, detection bound and impact — falls out of the rule.

anchoring is a trade, not a gain. It converts the log operator’s assumption from unbounded to bounded — the intended win, and the transparency-systems result of Section 2 made mechanical. But if a settlement layer is declared, the rule *adds* an assumption about that layer’s execution fidelity. A deep reorganization rolls back published attestation evidence. Anchoring is usually described as pure gain; in this calculus it visibly is not.

Table 2: The composition rules, as implemented in `src/mechanism.rs`. Fourteen rows, and the Δ column divides them three ways. Ten carry an unconditional ∞ : the declaration cannot express a bound at all, so it is a property of the mechanism rather than a default a deployment can override. Three carry no ∞ at all — the anchoring interval, the gossip propagation delay and the `tee_attestation` collateral refresh are declared durations, and a deployment must supply one. The remaining row, `anchoring`’s settlement assumption, is the only one where ∞ *is* an overridable default, applied when a settlement layer is declared without a finality bound.

Mechanism	Assumption (principal, capability)	Δ	Impact
<code>signing</code>	<code>signer, key_custody</code>	∞	SOUNDNESS
<code>hash_chain</code>	<code>log operator, append_only_monotonicity</code>	∞	SOUNDNESS
<code>anchoring</code>	<code>log operator, append_only_non_equivocation</code>	anchoring interval	SPLITVIEW
	<code>settlement layer (if declared), consensus_execution_fidelity</code>	declared finality, else ∞	SOUNDNESS
<code>gossip</code>	<code>each peer, view_synchronization</code>	propagation delay	SPLITVIEW
<code>witness_quorum</code>	<code>each witness, non_collusion_k_of_n</code>	∞	SOUNDNESS
<code>tee_attestation</code>	<code>endorser, silicon_and_microcode_integrity</code>	∞	SOUNDNESS
	<code>quoting enclave, quote_signing_honesty</code>	∞	SOUNDNESS
	<code>collateral authority, accurate_collateral_issuance</code>	collateral refresh	REVOCATION
	<code>reference-value provider, golden_value_correctness</code>	∞	SOUNDNESS
	<code>host, measurement_injection_resistance</code>	∞	SOUNDNESS
<code>zk_proof</code>	<code>ceremony, toxic_waste_destruction</code>	∞	SOUNDNESS
	<code>compiler, sound_arithmetization</code>	∞	SOUNDNESS
	<code>auditor, constraint_completeness</code>	∞	SOUNDNESS

witness_quorum names every witness, not *k* of them. The capability is `non_collusion_k_of_n` and it is attributed to each of the *n*. Two things are going on there and they are worth separating, because Section 6 turns on the difference.

Naming all *n* is not the arguable part. Drop any witness from the set and the threshold changes, so every one of them is load-bearing for the claim, and a trust set that listed only *k* of them would have to say which *k* — which the deployment cannot know in advance. All *n* belong.

The *count* is arguable. What a threshold rule asks a verifier to accept is one proposition, about the set: that no *k* of these *n* collude. Our assumption tuple is keyed by a principal, so it has no way to say “one assumption, about this set of *n*.” A set-valued assumption has nowhere to live except unrolled across its members, and so $|T|$ charges a quorum *n* times for something a verifier accepts once. That is a limitation of the representation rather than a claim about the world, we do not have a fix for it, and Section 6 shows what it costs. It is also where Section 7’s principal-independence problem bites hardest.

3.4 Delegation, and why it is a fixpoint

Mechanisms name principals directly. Deployments also contain delegation: *q speaks-for r* means that believing *r* commits you to trusting *q*. Four rules close the set:

1. A principal named by a mechanism supporting claim *P* is load-bearing for *P*.
2. If *r* is load-bearing for *P* and *q speaks-for r*, then *q* is load-bearing for *P*. (Delegation is transitive. This is the rule that diverges under naive SLD resolution.)
3. Every load-bearing principal contributes its own assumptions to the residual set.
4. Every delegate carries one assumption of its own: that its delegation is honest and unrevoked — capability `delegation_integrity`, with $\Delta = \infty$ and SOUNDNESS impact — whether or not it has mechanisms. Delegating to a party you did not otherwise depend on still puts that party in your trust set.

Rule 4 fires per delegation *edge*, tagged with the specific *r* it delegates through, not once per delegate. That detail was a bug before it was a design: a flat tag made every delegation assumption in a deployment byte-identical, so a principal delegating into two different mechanisms collapsed to one deduplicated assumption and a genuine shared dependency disappeared from the output.

3.5 Provenance, and why the comparison is order-independent

The fifth component of an assumption, the mechanism tag *m*, exists so that shared-dependency analysis can tell “two independent layers happen to name the same party” from “one layer named twice.” Without it, the same principal appearing under a TEE and under a ZK circuit would deduplicate into a single assumption and the most interesting finding in Section 5 would be invisible.

Because *m* participates in assumption identity, it also participates in comparison — and that is where we got it wrong. Tags were originally positional, of the form `kind#index`, index being the mechanism’s position in the deployment file. The consequence is easy to state and was embarrassing to find: **a deployment compared as incomparable to a reordering of itself**. Two files listing the same mechanisms in a different order produced different tags, hence different assumption tuples, hence two trust sets neither of which contained the other.

This matters beyond tidiness. The answer this paper offers to the completeness problem (Section 7) is to have a second author encode the same deployment independently and diff the two trust sets, treating any divergence as a finding. That experiment is worthless if the comparison reports divergence for two encodings that differ only in the order somebody typed the stanzas — it would produce a divergence every time and mean nothing.

Tags are now derived from *canonical content*, not position: a mechanism renders to a fixed-order string with list-valued fields sorted, and repeated identical declarations are distinguished by an ordinal among mechanisms sharing that content rather than by file position. So reordering distinct mechanisms cannot change any tag, while two genuinely repeated declarations stay distinct. The comparison is order-independent by construction, and the property is pinned by tests — one asserting that a deployment and its reordering compare EQUAL with an empty diff, one asserting that tags are a stable multiset under reordering, and one asserting that an `a, b, a` sequence assigns the second `a` ordinal 1 by content rather than 2 by position.

Removing positional sensitivity did not remove *lexical* sensitivity, and we found the second one the same way we found the first. Canonical rendering initially emitted each duration field as the text the author had typed, so `12h` and `720m` — the same 43 200 seconds, the same Δ in every derived assumption — produced different tags, and a deployment was once again incomparable to a rewrite of itself. The same held for `never` against `Never`, since the duration parser is case-insensitive and

the renderer was not. Duration fields are now rendered from the *parsed* value rather than the source text, which makes the rendering fallible and is the right trade: an unparseable duration is an error, not a mechanism whose identity is its own typo. We record this because the general lesson is not “do not index by position” but the weaker and more useful one: any part of a declaration that reaches a tag must reach it through the normalising function that defines what that part *means*, or the tag encodes spelling instead of content.

Two more constructions are refused rather than answered. A deployment declaring no mechanisms at all solves to the empty trust set, which takes $\Delta = \text{Bounded}(0)$ and sits below every other trust set under inclusion — so a two-line file would be reported as the most verifiable system in any comparison it entered. And two byte-identical mechanism stanzas would, under the ordinal rule above, be tagged distinctly and then read by the shared-dependency analysis as two independent layers, which two identical declarations never are. Both are rejected at validation. Neither is a case where a defensible answer exists and the tool merely picks the wrong one; in both, answering at all is the error.

The canonical rendering is unambiguous only because principal identifiers cannot contain the grammar’s delimiters. That is enforced once, when a deployment is validated, and it is the kind of invariant that a later convenience change could quietly break; the code says so at the point where it would be broken.

4 Why It Terminates

The design this implements proposed encoding the calculus as Datalog with tabled (SLG) resolution [20, 21], or a Soufflé equivalent [22], because real deployments contain circular delegation — A vouches for B vouches for C vouches for A — and a top-down proof search will chase that cycle forever. That reasoning is correct about SLD resolution and unnecessary here, and the difference is worth being precise about, because “we used a simpler method” is a claim a referee is entitled to check.

Here is the argument.

Proposition 1 (Termination). *For any finite deployment description, the residual trust set is reached in finitely many steps, regardless of cycles in the delegation graph.*

Argument. Fix a deployment with a finite set of declared principals $\mathcal{P}$, a finite list of mechanisms M , and a finite set of delegation edges E . Let A be the set of assumptions that can ever be derived. A is finite: each mechanism $m \in M$ contributes $|\mathcal{A}(m)|$ assumptions, a constant for every rule except `gossip` and `witness_quorum`, where it is the number of declared peers or witnesses, and `anchoring`, where it is one or two according to whether a settlement layer is declared; and each delegation edge contributes exactly one `delegation_integrity` assumption tagged with its target. Each of those is finite, so $|A| \leq \sum_{m \in M} |\mathcal{A}(m)| + |E|$ is finite.

The residual set is the least fixpoint of a monotone operator on $(\mathcal{P}(A), \subseteq)$. Every rule is monotone — no rule ever removes an assumption or a load-bearing fact — and the lattice has height $|A|$, which is finite. A monotone operator on a complete lattice has a least fixpoint [23], and on a lattice of finite height Kleene iteration reaches it after at most $|A|$ rounds. Cycles in the `speaks-for` relation cannot change this, because rule 2 derives facts in the finite domain $\mathcal{C} \times \mathcal{P}$ rather than generating new terms: a cycle simply means the same finite set of facts is re-derivable by more than one path, and re-deriving a fact already present adds nothing and terminates the round.

The latency lattice takes no part in this. It has infinite height, and if the solver iterated in it the argument above would not go through; it does not. Δ is folded over the converged trust set afterwards, over finitely many members, so it terminates for the trivial reason. $\square$

The reason tabling is unnecessary, stated plainly: SLD resolution diverges on cyclic data because it is a top-down search that generates goals, and nothing stops it from generating the same goal forever. Tabling fixes that by remembering goals it has already tried. But we are not doing a top-down proof search. We evaluate bottom-up over ground facts in a finite domain, so the thing tabling remembers is the thing bottom-up evaluation computes anyway. The machinery solves a problem this formulation does not have. In practice we express the rules in **ascent** [24], a Datalog embedded in Rust with support for lattice-valued relations, which does semi-naive bottom-up evaluation; the lattice-valued relation is exactly the residual trust set, with union as its join.

This is not only an argument. A test in the artifact builds a delegation cycle through 300 principals, spanning two mechanism layers, and solves it: every principal comes out load-bearing, as the cycle requires, and the solve — the clock starts after parsing and validation, so it is the fixpoint that is timed — returns in well under a second on a laptop in an unoptimized build. We report that as evidence of termination, not as a performance result; we have not characterized how the runtime grows, and Section 7 lists the complexity bound as unverified.

5 Does It Actually Work?

Every table and figure in this section is tool output. `scripts/regen-results.sh` runs **parallax** over the encoded deployments, writes each one into `results/`, and this section reads them in from there. Continuous integration reruns the script and fails the build if a committed result disagrees with the code that produced it, so a number *in a table* here cannot outlive that code.

The prose around the tables is a different matter, and worth being exact about rather than waving at. Where a sentence below restates a figure from a table — how many rows are undetectable, how many pairs are N/A — that sentence was typed, and a typed restatement can drift away from the table it sits next to. Each one that could is pinned by a test in the artifact; `paper/README.md` names them. That is a weaker guarantee than the tables have, and we would rather say so than imply the whole section is generated.

Five deployments are encoded and shipped with the artifact: Σ_1 , a software-only host with signed measurement logs; Σ_2 , Intel TDX with ECDSA quotes and PCS collateral; Σ_3 , a 5-of-7 witness quorum over a transparency log; Σ_4 , a Groth16 rollup with an on-chain verifier; and Σ_5 , a hybrid combining a TEE with a ZK proof. A sixth file ships as well: a variant of Σ_5 whose two layers use genuinely separate build pipelines, which exists only as the negative control for Section 5.3. All of them were written by us. Nothing here has been validated against somebody else’s production deployment, and these examples are the entire evidence base for every number in this section.

5.1 Reproducing the hand-derived TDX set

The first success criterion, from the original design, is that the tool produces the five parties of Table 1 from the Σ_2 description alone. It does. Table 3 is what **parallax** prints, read into this document from the file the tool wrote. Set it beside Table 1, which was derived by hand before the tool existed: the same five parties, the same five capabilities, the same one bounded row.

It is worth being exact about what that does and does not demonstrate, because the criterion as originally worded — that the five emerge “without those parties being named in the encoding” — is not what happens.

The five parties *are* named individually in `examples/sigma2-tdx.toml`, as the endorser, quoting enclave, collateral authority, reference-value provider and host fields of a single `tee_attestation` stanza. What the deployment file does not contain is any statement that those five constitute a trust set, or what each of them must be trusted *for*. That is what the rule supplies: each party’s

Table 3: The residual trust set `parallax` computes for Σ_2 , from `parallax solve --format latex`. The `tee_attestation` rule supplies every capability, every impact, and *which* row is bounded — four of the five are ∞ because the rule says the endorser, the quoting enclave, the host and the reference-value provider have no detection mechanism. The deployment file supplies the five identifiers and one number: the collateral refresh interval, which is the bound in the one row that has one.

Principal	Capability assumed	Detectable in	Impact
did:web:cloud.example.com	measurement_injection_ resistance	∞	SOUNDNESS
did:web:intel.com	silicon_and_microcode_ integrity	∞	SOUNDNESS
did:web:pcs.intel.com	accurate_collateral_issuance	43200s	REVOCATION
did:web:rvp.example.org	golden_value_correctness	∞	SOUNDNESS
urn:qe:tdx	quote_signing_honesty	∞	SOUNDNESS

capability, its detection bound, its failure impact, and — the part that is actually a claim about the deployment rather than about the rule — that exactly these five and no others are load-bearing for the claim. A sixth principal declared in the file but not reachable from any mechanism or delegation does not appear in the output.

So the honest statement of the result is narrower than the original criterion: the tool derives the *content* of Table 1 from a mechanism kind and a set of field values, not the *identities* of the parties. The identities came from whoever wrote the file, which is exactly the completeness problem of Section 7 showing up in the success criterion.

5.2 The comparison matrix

Table 4 is assembled by `scripts/regen-results.sh`. The four INCOMPARABLE cells are the tool’s own output, from `parallax compare`. The eight N/A cells are not: the script reads the `claim` field out of each deployment file, sees that the two differ, and writes the N/A row without invoking the tool at all — because `parallax compare` refuses the pair and exits non-zero. Either way the file is generated rather than typed, and continuous integration regenerates it and fails on a diff, so it cannot drift from the code.

Every pair the tool will rank at all — that is, every pair attesting the same proposition — comes out INCOMPARABLE. If that holds up, it is the headline: under set inclusion there is no partial order over these designs, so no ordinal tier ladder can be sound, and conformance should publish a trust set rather than a tier.

5.3 The shared dependency

The second success criterion is a non-obvious finding. Σ_5 layers a hardware TEE under a zero-knowledge proof and is the shape of design that gets described as defence in depth. The tool reports that both layers rest on the same build pipeline: it supplies the reference values the TEE measurement is compared against *and* compiles the circuit whose constraints the proof relies on. One build-pipeline compromise corrupts both simultaneously. The two layers are not independent.

Figure 1 is the run, verbatim. The block at the bottom is the finding: one principal, reached under two different mechanism tags, with the capability each layer needs from it. The long parenthesised strings are the canonical mechanism tags of Section 3.5 — they are what makes the two layers distinguishable, and they are why the same party appearing twice does not deduplicate into one assumption.

Table 4: Every ordered pair among the four non-hybrid deployments Σ_1 – Σ_4 under set inclusion. Pairs attesting different propositions are marked N/A rather than omitted: **parallax** refuses to rank verifiability across two different claims, and a truncated matrix would be indistinguishable from a broken run. Eight of the twelve ordered pairs are N/A, so the table rests on two unordered comparisons — Σ_1/Σ_3 and Σ_2/Σ_4 .

Deployment A	Deployment B	Relation under set inclusion
sigma1-software	sigma2-tdx	N/A — different claims (<code>measurement_valid</code> vs <code>execution_valid</code>)
sigma1-software	sigma3-quorum	Incomparable
sigma1-software	sigma4-zk	N/A — different claims (<code>measurement_valid</code> vs <code>execution_valid</code>)
sigma2-tdx	sigma1-software	N/A — different claims (<code>execution_valid</code> vs <code>measurement_valid</code>)
sigma2-tdx	sigma3-quorum	N/A — different claims (<code>execution_valid</code> vs <code>measurement_valid</code>)
sigma2-tdx	sigma4-zk	Incomparable
sigma3-quorum	sigma1-software	Incomparable
sigma3-quorum	sigma2-tdx	N/A — different claims (<code>measurement_valid</code> vs <code>execution_valid</code>)
sigma3-quorum	sigma4-zk	N/A — different claims (<code>measurement_valid</code> vs <code>execution_valid</code>)
sigma4-zk	sigma1-software	N/A — different claims (<code>execution_valid</code> vs <code>measurement_valid</code>)
sigma4-zk	sigma2-tdx	Incomparable
sigma4-zk	sigma3-quorum	N/A — different claims (<code>execution_valid</code> vs <code>measurement_valid</code>)

We should be careful about how much credit this deserves. We constructed Σ_5 , so the tool found something we put there. The claim is that the analysis finds it from the description — the shared principal is not remarked on anywhere in the file — and that a matched variant with genuinely separate pipelines reports nothing, so the check is not simply always firing. Whether it finds such things in deployments nobody constructed is untested.

5.4 Detection latency and policy

Table 5 is all five deployments at a glance: how many assumptions each rests on, how many distinct parties those assumptions name, how many of them nothing would ever report a violation of, and the composed Δ .

The last column is the same for every row, and that is the finding rather than a defect in the table. Δ composes as a join over the whole trust set (Section 3), so one member with no detection mechanism takes the entire system to ∞ , and the *Undetectable* column is nonzero in every row. That includes Σ_3 , the witness quorum with an anchored log and gossip — the design in this set that exists specifically to make misbehaviour visible. Its anchoring interval and its gossip propagation delay are real bounds on real parties, and they are invisible in the composed figure because the seven witnesses sitting underneath them are not detectable at all. Whether the max is the right composition is Section 7’s question; what the column shows is that under the composition

```

PRINCIPAL CAPABILITY DETECT IMPACT
did:web:auditor.example constraint_completeness never Soundness
did:web:buildco.example golden_value_correctness never Soundness
did:web:buildco.example sound_arithmetization never Soundness
did:web:ceremony.example toxic_waste_destruction never Soundness
did:web:cloud.example.com measurement_injection_resistance never Soundness
did:web:intel.com silicon_and_microcode_integrity never Soundness
did:web:pcs.intel.com accurate_collateral_issuance 43200s Revocation
urn:qe:tdx quote_signing_honesty never Soundness

8 assumptions, 7 principals

SHARED DEPENDENCIES -layers that are not independent:
  did:web:buildco.example
    tee_attestation
      tee_attestation(endorser=did:web:intel.com,quoting_enclave=urn:qe:tdx,collateral_authority=did:web:pcs.
        intel.com,collateral_refresh=43200s,reference_values=did:web:buildco.example,host=did:web:cloud.
        example.com)#0
      golden_value_correctness (never)
    zk_proof
      zk_proof(ceremony=did:web:ceremony.example,compiler=did:web:buildco.example,auditor=did:web:auditor.
        example)#0
      sound_arithmetization (never)

```

Figure 1: `parallax solve sigma5-hybrid.toml --shared`, as the tool prints it. The build pipeline is named once by the TEE stanza, as the reference-value provider, and once by the ZK stanza, as the circuit compiler. Nothing in the deployment file remarks on that.

Table 5: Every encoded deployment, from `parallax tiers --format tex-summary`. *Undetectable* counts assumptions with $\Delta = \infty$. Principals is lower than $|T|$ only for Σ_5 , where one party is trusted for two different capabilities — which is the shared dependency of Section 5.3 showing up as a number.

Deployment	Claim	$ T $	Principals	Undetectable	Composed Δ
sigma1-software	measurement_valid	3	3	3	∞
sigma2-tdx	execution_valid	5	5	4	∞
sigma3-quorum	measurement_valid	11	11	7	∞
sigma4-zk	execution_valid	5	5	3	∞
sigma5-hybrid	execution_valid	8	7	7	∞

we implemented, no deployment we encoded reaches a finite bound.

That is also what makes a strict policy useless as a filter here. Table 6 checks each manifest against `examples/policy-strict.toml`, a relying party that will not accept any undetectable assumption and wants everything else detectable inside a day. Every deployment fails, and the violation count is just the *Undetectable* column of Table 5 again — the 24-hour bound never binds, because the only assumptions with finite bounds at all are well inside it.

A policy that rejects every candidate is not yet a useful policy, and we do not read this as showing that strict policies are wrong. It shows that the strict clause everybody reaches for first — “no silent failure modes” — is not satisfiable by any of the designs we know how to encode, which is the same fact as Table 1’s fourth column stated as a machine-checkable rule. What a relying party would actually write is a policy that names which undetectable assumptions it will carry and why. The tool has no way to express that today, and Section 7 lists it as open.

Table 6: Each manifest against `examples/policy-strict.toml` (`forbid_undetectable = true`, `max_detection_latency = "24h"`), from `parallax check`. `parallax` exits 1 on violations, so this is a gate a relying party could run in CI. On this evidence it is a gate that rejects everything.

Deployment	Verdict	Violations
<code>sigma1-software</code>	violates the policy	3
<code>sigma2-tdx</code>	violates the policy	4
<code>sigma3-quorum</code>	violates the policy	7
<code>sigma4-zk</code>	violates the policy	3
<code>sigma5-hybrid</code>	violates the policy	7

6 Can the Ladder Be Repaired?

A tier ladder is an ordinal claim. Tier 3 sits above Tier 2, which means there has to be some way of taking two deployments and saying which is more verifiable. We can think of four candidates, and they are ours: the list comes from the design study this paper implements, not from a survey of the literature, and we do not claim it is exhaustive. If there is a fifth that works, this section does not rule it out. `parallax tiers` runs each of the four over Σ_1 – Σ_4 — the same four the comparison matrix covers — and reports what each does and whether the result is an order you could put numbers on. Table 7 is that output.

None of the four survives. The reason the experiment was worth running is that they do not fail the same way.

Table 7: The four candidate orderings applied to Σ_1 – Σ_4 , from `parallax tiers --format tex-orderings`. The middle column reads left to right as most verifiable to least, under the candidate’s own key. Σ_5 is excluded because it is a hybrid built to carry a shared dependency rather than a fifth design competing for a rung; the comparison matrix leaves it out for the same reason.

Candidate ordering	Ranking it produces, most verifiable first	Verdict
Cardinality $ T $	<code>sigma1-software</code> (3) < <code>sigma2-tdx</code> (5) = <code>sigma4-zk</code> (5) < <code>sigma3-quorum</code> (11)	No total order: <code>sigma2-tdx</code> and <code>sigma4-zk</code> tie at 5.
Set inclusion	—	No total order: of 6 unordered pairs, 4 were refused for attesting different claims and 2 came out incomparable; 0 were strictly ranked and 0 were equal.
Composed Δ	<code>sigma1-software</code> (never) = <code>sigma2-tdx</code> (never) = <code>sigma3-quorum</code> (never) = <code>sigma4-zk</code> (never)	No order: every deployment scores never, so this key separates nothing.
Collusion cost	—	Not computable: a deployment description names principals but not what it costs an adversary to corrupt them jointly, which is a property of the adversary and the world rather than of the description.

Cardinality ranks the systems backwards. Count the assumptions and put the smallest number first. On these four that puts Σ_1 at the top: a software-only host that signs its own measurement log and hash-chains it, with no hardware root, no witnesses, and nothing anybody outside the host can check. It puts Σ_3 at the bottom: a 5-of-7 witness quorum over a log that is anchored and gossiped. Nobody using the word “verifiable” means that. The quorum is doing more work to be checkable and the count charges it for every witness it added.

The problem is not that the metric is crude. It is that it rewards the wrong thing. $|T|$ measures how much of a trust structure somebody wrote down, and under this ordering the way to climb is to write down less — which is the completeness problem of Section 7 turned into an incentive. Cardinality also fails on its own terms before you get to any of that: Σ_2 and Σ_4 carry the same number of assumptions and tie, so it is not a total order even where it does rank.

Set inclusion ranks almost nothing. This is the order the tool already implements, and nothing about it is wrong — inclusion is a real order and it says something true whenever it applies. It just hardly ever applies. Most of the unordered pairs among these four are refused outright, because the two sides attest different propositions and asking which of two different claims is more verifiable is not a question. Every pair that survives that filter comes out incomparable: each trust set holds an assumption the other does not. Nothing at all gets ranked. That is Table 4 restated as an ordering question, and the answer is that on this evidence the partial order is entirely partial.

Composed detection latency ranks everything the same. Order by how fast the system would notice. This one is not wrong either; it is empty. The composed Δ is ∞ for all four, including the anchored, gossiped quorum, for the reason Section 5.4 gives: Δ is a join over the whole trust set, and the *undetectable* column of Table 5 is nonzero in every row. One silent assumption takes the whole system to ∞ , and none of these designs gets to zero silent assumptions. A key that is constant across everything you want to rank separates nothing.

Collusion cost is not in the input. Order by what it would cost an adversary to corrupt enough parties at once. This is the one people reach for when the others fail, and it is the one *parallax* refuses to score. A deployment description names principals. It does not say who owns them, what jurisdictions they sit in, whose budget could reach them, or whether two identifiers with different names belong to two companies or one — which is the principal-independence problem of Section 7, and we do not know how to compute it either. Those are facts about the world and about a particular adversary, and the same two designs will rank in opposite orders for a domestic enterprise and for a cross-border protocol. The tool reports collusion cost as not computable and says why, rather than printing a number it would have made up. A made-up number inside a trust manifest is the exact failure this paper exists to object to.

What follows from that. Four candidates, four independent failures: one ranks the systems backwards, one ranks almost nothing, one ranks everything the same, and one is not derivable from the input at all. We have no repaired ladder to offer in place of the broken one, and having run this we do not expect one to exist.

What we have instead is what the original design already proposed, and this is the result that supports it: conformance should publish a trust manifest — the parties, the capabilities, the detection bounds, the impacts — and let a relying party evaluate it against a local policy, rather than claim a tier. A tier is a single number that asserts an ordering. A manifest asserts no ordering, which on this evidence is the honest thing for it to assert. Section 5.4 shows the policy

check running and shows it rejecting all five deployments, so this is a direction rather than a finished answer; Section 7 lists what it leaves open, including how an unfamiliar manifest gets evaluated at all.

What this rests on. Four deployments, all of which we wrote. The experiment is a computation over four trust sets the tool derived from four files we authored, and it cannot show that no ordering exists. It shows that these four candidates fail on these four deployments. Two of the four failures look structural rather than accidental — cardinality charges a deployment for every party it names, and collusion cost is absent from the input no matter what the input is — and we would expect those two to reproduce anywhere. The first comes with a caveat about our encoding that we set out below. The other two are contingent on the evidence. Set inclusion is empty here because these four trust sets happen to be pairwise disjoint or claim-mismatched; a fifth deployment somebody else wrote could produce a genuine subset pair. Composed Δ is constant here because no design we encoded eliminates its last silent assumption; a design that did would separate immediately. Neither of those experiments has been run.

One of the two we called structural needs a caveat of its own, and it is the first thing a referee should push on.

Cardinality puts Σ_3 last only because `witness_quorum` attributes `non_collusion_k_of_n` to each of the n witnesses rather than once to the mechanism. Suppose you reject that and count one assumption per declared mechanism instead. The ranking does not merely lose its shape; it turns over. Σ_2 declares one mechanism, Σ_4 declares two, and Σ_1 and Σ_3 declare three apiece. The hardware-attested deployment comes first and the software-only host ties the witness quorum for last — which is the opposite of the paragraph above, not a tie-break away from it.

We had this wrong in an earlier draft, in this paragraph, which is the paragraph about what we are not entitled to claim. It said Σ_1 has the fewest mechanisms as well as the fewest assumptions. It has the most. The numbers above now come from the deployment files rather than from us.

There is a narrower objection that does leave Σ_1 on top: collapse only the quorum’s n non-collusion assumptions and leave every other rule alone. But nothing in “one assumption per mechanism” singles that reading out, and we should not get to pick the version of the objection that flatters us.

So how much of “cardinality ranks the systems backwards” survives? The part that does not depend on the convention is about *principals* rather than assumptions. Every one of Σ_3 ’s witnesses is load-bearing on any reading — drop one and the threshold changes — so the principals column of Table 5 is not an encoding artifact, and it ranks Σ_3 last too. What *is* an artifact is the assumption count: Section 3 explains that our tuple is keyed by a principal and therefore cannot express one assumption about a set of n , so a threshold gets charged n times for a proposition a verifier accepts once.

That leaves us saying less than we would like, and we would rather say the smaller true thing. Cardinality ranks a deployment worse for naming more parties, and being checkable by strangers generally means naming more parties; we think that much holds anywhere. Whether it ranks these four particular deployments backwards depends on a decomposition we chose, and a reader who chooses differently gets a different — and, under the obvious alternative, an even less defensible — answer. Neither reading rescues the ladder, which is the section’s point, but only one of them is ours to assert.

7 What We Still Don’t Know

7.1 The completeness problem, which is the real objection

A solver computes over the dependencies that were *encoded*. It cannot discover one that was not.

This is the thing to worry about, and it is worse than an ordinary limitation, because the failure mode is not silence but confidence. A tool that prints a tidy five-element trust set while a sixth dependency exists off-model has converted an open question into a wrong answer, and it has done so in a format designed to be trusted. That is worse than no tool. If you take one thing from this paper, we would rather it be this sentence than the incomparability result.

We do not have a solution. We have three partial answers, none satisfying:

1. **Independent encoding and diff.** Have a second author encode the same deployment without seeing the first encoding, and diff the trust sets. Any divergence is a finding about the calculus rather than a bug to fix quietly. The tool supports this — `parallax diff` exits non-zero when two trust sets differ, so it can be a CI gate — and Section 3.5 explains why the comparison had to be made order-independent before the experiment could mean anything. We have not run it with a genuinely independent author. Until we do, this is a plan, not a result. Note also what it can and cannot detect: it finds dependencies one author saw and the other missed. It is blind to a dependency that neither saw, which is the case we are most afraid of.
2. **Adversarial review of the description.** Treat the deployment file as the artifact under attack rather than the tool. We have not built a procedure for this.
3. **Printing the caveat with every output.** A manifest should carry, in band, the statement that it enumerates modelled dependencies only, so that the caveat travels with the artifact instead of living in a paper nobody reads alongside it. The manifests the tool emits today *do not* carry such a field; this is the weakest of the three answers, the only one that is free, and we have not yet implemented it.

7.2 The rules are not proved sound

Table 2 is a set of design decisions, not theorems. Nothing here proves that if the parties in T are honest then P actually holds, against an operational model in which a named principal may deviate arbitrarily. Soundness is where the artifact’s value ultimately has to come from, and it is future work. What we have is a mechanization of rules that agree with a careful hand analysis on the cases we checked.

7.3 Principal independence is asserted, never computed

A quorum rule reduces to a non-collusion predicate over named parties, and nothing in the calculus prevents five of seven “independent” witnesses from sharing a jurisdiction, a cloud region, a corporate parent, or a hardware supplier. The tool finds shared *principals* — that is the Σ_5 result — but a shared principal is the easy case, because at least both layers named the same identifier. Two distinct identifiers that happen to be subsidiaries of the same company are invisible to it.

Whether independence can be computed from a description at all, or must be asserted by somebody who knows the world, is open and consequential. Our suspicion is that it must be asserted, in which case the honest design is for the manifest to record who asserted it.

7.4 Whether Δ should compose as a max

We compose detection latency as a join that takes the worse bound, with ∞ absorbing. The argument for it is that a system is only as detectable as its least detectable part, which is the right thing to tell somebody deciding whether to rely on the evidence. The arguments against it are real:

Two independent monitors watching the same party do not obviously give you the worse of their two latencies; you might argue for the better, since either one detecting is enough. A chain of detection steps — an anchor that must be published, then gossiped, then noticed — might argue for a sum. And a judgment carries a validity window that the composed Δ does not represent: what a 12h collateral bound means when the quote is checked 13h later is a question we currently answer by degenerating to the unmonitored case, which is a choice and not a derivation.

Max is defensible and it is what is implemented. We do not claim it is derived.

7.5 The evidence base is five deployments, all ours

Every result in Sections 5 and 6 comes from five deployments we wrote, plus a sixth file that exists only as a negative control for one of them. The tool reproduces a trust set we had already derived by hand — which is evidence it implements what we intended, and weaker evidence that what we intended is right — and finds a shared dependency in a deployment we constructed to contain one. Neither of those is validation against reality. The complexity claim from the original design, that computing $T(\Sigma)$ is polynomial in principals $\times$ credentials, is likewise unverified: we have a termination argument (Section 4) and one 300-node example, not a bound.

7.6 If the ladder goes, what replaces it

Supposing incomparability holds and conformance publishes a manifest instead of a tier, three questions open that we do not answer. How does a relying party with a local policy evaluate an unfamiliar manifest? The tool has a policy checker, which is a start and not an answer. How are principals named stably enough to compare manifests written by different people — our examples use DIDs, which pushes the problem somewhere else rather than solving it. And what stops manifest inflation, where a vendor lists many trivial assumptions to look rigorous? A trust set is only useful if listing more assumptions is not rewarded.

8 Conclusion

The claim that hardware attestation leaves nobody to trust is not a small overstatement; it is wrong in a way that changes what a conformance claim means. Believing one Intel TDX measurement requires accepting five distinct parties, and for four of them there is no mechanism anywhere in the system that would ever report a violation. That is worth writing down, and the way to make sure it gets written down is to compute it rather than argue about it.

The calculus in this paper is small: a judgment with a trust set and a detection bound, a composition rule per mechanism, two lattices, and a least fixpoint. Its termination on the case everyone worries about — circular delegation — follows from finite lattice height and needs none of the machinery originally proposed for it. Its output on the deployments we encoded suggests that residual trust sets are incomparable rather than ordered, which would mean the ladder is the wrong abstraction and conformance should publish a set. We then tried the four orderings that might have rescued the ladder, and none of them works: one ranks the systems backwards, one ranks

almost nothing, one ranks everything the same, and one asks for a fact no deployment description contains.

And the tool cannot see what nobody encoded. We would rather that be the sentence people remember than the incomparability result, because a residual trust set is a claim about what you depend on, and a confident claim of that kind is exactly the thing this paper began by objecting to.

Acknowledgments

Prepared within the Proof-of-Control Initiative of the Advanced AI Society. The Initiative’s working groups, board and advisory board are credited separately; contributions to this manuscript are the basis for authorship.

References

- [1] Advanced AI Society, Proof-of-Control Initiative. Proof-of-control: An open standard for runtime verifiability and cryptographic oversight in autonomous ai execution. Working draft, 2026. [verify] repository URL and version at submission time.
- [2] Jim Schwoebel. **parallax**: Computing the residual trust set of an attestation deployment. Apache-2.0 licensed software, 2026. [verify] public repository URL at submission time.
- [3] Intel Corporation. Intel trust domain extensions (Intel TDX) module architecture specification. <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>, 2023. [verify] document revision and date.
- [4] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, Internet Engineering Task Force, 2023.
- [5] Martín Abadi, Michael Burrows, Butler Lampson, and Gordon Plotkin. A calculus for access control in distributed systems. *ACM Transactions on Programming Languages and Systems*, 15(4):706–734, 1993.
- [6] Butler Lampson, Martín Abadi, Michael Burrows, and Edward Wobber. Authentication in distributed systems: Theory and practice. *ACM Transactions on Computer Systems*, 10(4):265–310, 1992.
- [7] Fred B. Schneider, Kevin Walsh, and Emin Gün Sirer. Nexus authorization logic (NAL): Design rationale and applications. *ACM Transactions on Information and System Security*, 14(1):8:1–8:28, 2011.
- [8] Moritz Y. Becker, Cédric Fournet, and Andrew D. Gordon. SecPAL: Design and semantics of a decentralized authorization language. *Journal of Computer Security*, 18(4):619–665, 2010.
- [9] Yuri Gurevich and Itay Neeman. DKAL: Distributed-knowledge authorization language. In *21st IEEE Computer Security Foundations Symposium (CSF)*, pages 149–162, 2008.
- [10] John DeTreville. Binder, a logic-based security language. In *IEEE Symposium on Security and Privacy*, pages 105–113, 2002.

- [11] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013.
- [12] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, Internet Engineering Task Force, 2021.
- [13] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium*, pages 383–398, 2015.
- [14] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *27th USENIX Security Symposium*, pages 991–1008, 2018.
- [15] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. SGAXe: How SGX fails in practice. <https://sgaxe.com/files/SGAXe.pdf>, 2020. Technical report.
- [16] Kit Murdock, David Oswald, Flavio D. Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. Plundervolt: Software-based fault injection attacks against Intel SGX. In *IEEE Symposium on Security and Privacy*, pages 1466–1482, 2020.
- [17] Pietro Borrello, Andreas Kogler, Martin Schwarzl, Moritz Lipp, Daniel Gruss, and Michael Schwarz. $\mathcal{A}$ PIC leak: Architecturally leaking uninitialized data from the microarchitecture. In *31st USENIX Security Symposium*, pages 3917–3934, 2022.
- [18] Victor Costan and Srinivas Devadas. Intel SGX explained. IACR Cryptology ePrint Archive, Report 2016/086, 2016.
- [19] Ernie Brickell, Jan Camenisch, and Liqun Chen. Direct anonymous attestation. In *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*, pages 132–145, 2004.
- [20] Weidong Chen and David S. Warren. Tabled evaluation with delaying for general logic programs. *Journal of the ACM*, 43(1):20–74, 1996.
- [21] Terrance Swift and David S. Warren. XSB: Extending Prolog with tabled logic programming. *Theory and Practice of Logic Programming*, 12(1-2):157–187, 2012.
- [22] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. Soufflé: On synthesis of program analyzers. In *Computer Aided Verification (CAV)*, pages 422–430, 2016.
- [23] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955.
- [24] Arash Sahebdlamri, Thomas Gilray, and Kristopher Micinski. Seamless deductive inference via macros. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction (CC)*, pages 77–88, 2022. [verify] pages and proceedings details.